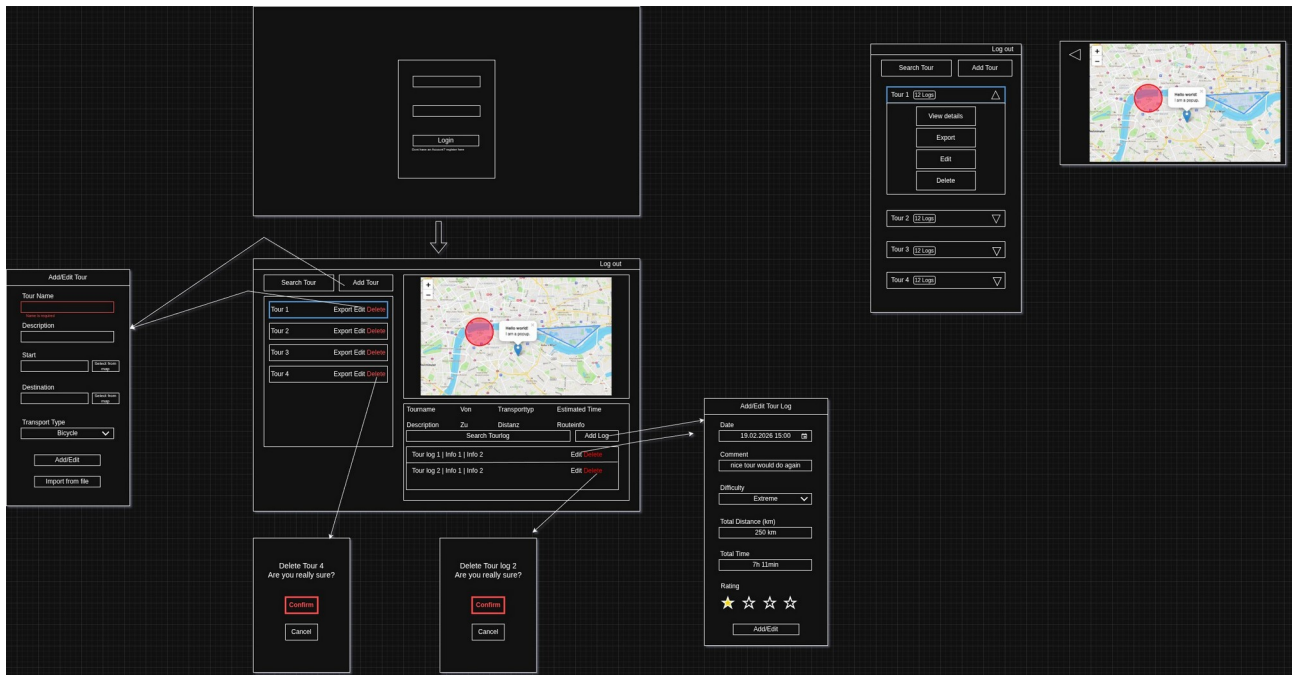


## Describes app architecture (layers, layer contents/functionality, class diagrams)

The app is build with an layered architecture. First you have the controller that accepts the requests. The Controller then uses the corresponding service which process the request data and uses the repository to create, get, delete or update data.

## Describes UX (include wireframes)



The UI is split into two sections. On the left you have a list with the Tours and on the right is the map where you can see the tour route and underneath the map is a list with TourLogs.

When you create or update a Tour or TourLog a Modal gets opened where the user can define the details. In case of the Tour you can define the tour start and endpoint through a click on the map when you click the button “Select from map”.

## **Describes library decisions (where applicable), lessons learned**

The project uses Spring Boot for the backend because it fits the REST API, layered service structure, validation, security, and JPA persistence well. Spring Data JPA reduces repository boilerplate, PostgreSQL is used as the main database, and H2 supports lightweight testing. Lombok keeps entities and services shorter. On the frontend, Angular was chosen for structured components, routing, forms, and services. Tailwind/DaisyUI handles styling and themes, Leaflet/OpenStreetMap provides map interaction, ngx-toastr is used for user feedback, and Docker Compose simplifies running backend plus database together.

## **Describes implemented design pattern**

The main implemented pattern is the layered architecture. Controllers receive HTTP requests, services contain business logic, repositories handle database access, and entities/DTOs separate persistence data from API data. Spring's dependency injection connects these layers through services and repositories with help of the `@Autowired` annotation.

## **Describes unit testing decisions**

The test strategy was that the main functionalities of a service gets tested. Like for the TourService the TourCreation, Tour Update, Tour Delete.

For the UserService only the registration was tested, because the login is implicitly tested through every test of the TourService or TourLogService as they need the userId to create, delete or update a Tour or TourLog.

## **Describes unique feature**

The unique feature is, that you can change the theme of the frontend with a ThemeController of DaisyUI. To change the theme you must only click on dropdown on the right in the navbar and select your preferred theme.

Additionally you can select the tour startpoint and endpoint on the map through a click on the button "Select from map" in the tour creation modal. After selecting a point on the map the Latitude and Longitude of this point is displayed in the modal on the left of the button.